로컬비서 (LocalBiseo) v0.1.0 소스코드 출력물

- 대상: 직접 작성한 소스 파일 81개, 9,894줄
- 제외: 오픈소스 라이브러리, AI 모델, 글꼴, 자동 생성 파일, 테스트 코드
- 각 쪽 위에 파일 경로, 각 줄 앞에 줄 번호를 표시했다.
- 발췌본: 앞 25쪽과 뒤 25쪽

```rust
//! 화면 캡처와 클립보드 감시.

use std::path::PathBuf;
use std::time::{Duration, SystemTime, UNIX_EPOCH};

use tauri::{AppHandle, Emitter, Manager};

/// 주 모니터를 캡처해 임시 폴더에 PNG로 저장하고 경로를 돌려준다.
/// 우리 창이 찍히지 않도록 잠깐 숨겼다가 다시 띄운다.
pub fn capture_primary(app: &AppHandle) -> Result<String, String> {
    let window = app.get_webview_window("main");
    if let Some(w) = &window {
        let _ = w.hide();
    }
    std::thread::sleep(Duration::from_millis(250));
    let result = (|| {
        let monitors = xcap::Monitor::all().map_err(|e| format!("화면을 찾지 못했어요: {e}"))?;
        let monitor = monitors
            .iter()
            .find(|m| m.is_primary().unwrap_or(false))
            .or_else(|| monitors.first())
            .ok_or("화면을 찾지 못했어요.")?;
        let image = monitor.capture_image().map_err(|e| format!("화면을 캡처하지 못했어요: {e}"))?;
        let stamp = SystemTime::now().duration_since(UNIX_EPOCH).map(|d| d.as_millis()).unwrap_or(0);
        let path: PathBuf = std::env::temp_dir().join(format!("localbiseo-capture-{stamp}.png"));
        image.save(&path).map_err(|e| format!("캡처를 저장하지 못했어요: {e}"))?;
        Ok(path.to_string_lossy().into_owned())
    })();
    if let Some(w) = &window {
        let _ = w.show();
        let _ = w.set_focus();
    }
    result
}

/// 클립보드 글자가 바뀌면 화면에 `clipboard-text` 이벤트로 알린다.
/// 저장할지 말지(설정, 민감정보 처리)는 엔진이 정한다.
pub fn watch_clipboard(app: AppHandle) {
    std::thread::spawn(move || {
        let mut board = match arboard::Clipboard::new() {
            Ok(b) => b,
            Err(_) => return, // 클립보드를 쓸 수 없는 환경
        };
        let mut last = board.get_text().unwrap_or_default();
        loop {
            std::thread::sleep(Duration::from_millis(800));
            if let Ok(text) = board.get_text() {
                if text != last {
                    last = text.clone();
                    if !text.trim().is_empty() {
                        let _ = app.emit("clipboard-text", text);
                    }
                }
            }
        }
    });
}
```

```rust
//! Python 엔진을 자식 프로세스로 띄우고 주소 · 토큰을 화면에 알려준다.
//!
//! 엔진은 준비되면 표준출력에 `LOCALBISEO_READY <port>`를 찍는다.
//! 표준입력을 열어 둔 채로 두면, 앱이 죽어도 파이프가 닫혀 엔진이 같이 끝난다.

use std::io::{BufRead, BufReader};
use std::path::PathBuf;
use std::process::{Child, Command, Stdio};
use std::sync::{mpsc, Arc, Condvar, Mutex};
use std::time::Duration;

use serde::Serialize;
use tauri::{AppHandle, Manager};

#[derive(Clone, Serialize)]
pub struct EngineInfo {
    pub url: String,
    pub token: String,
}

#[derive(Default)]
pub struct EngineState {
    ready: Mutex<Option<Result<EngineInfo, String>>>,
    signal: Condvar,
    child: Mutex<Option<Child>>,
}

pub type SharedEngine = Arc<EngineState>;

impl EngineState {
    pub fn wait(&self, timeout: Duration) -> Result<EngineInfo, String> {
        let guard = self.ready.lock().unwrap();
        let (guard, _) = self
            .signal
            .wait_timeout_while(guard, timeout, |r| r.is_none())
            .unwrap();
        guard
            .clone()
            .unwrap_or_else(|| Err("엔진이 시간 안에 준비되지 않았어요.".into()))
    }

    fn finish(&self, result: Result<EngineInfo, String>) {
        *self.ready.lock().unwrap() = Some(result);
        self.signal.notify_all();
    }

    pub fn kill(&self) {
        if let Some(mut child) = self.child.lock().unwrap().take() {
            let _ = child.kill();
            let _ = child.wait();
        }
    }
}

/// 엔진 실행 명령. 배포판은 리소스의 `engine/localbiseo-engine`, 개발 중에는 `python -m localbiseo`.
fn engine_command(app: &AppHandle) -> Command {
    let exe_name = if cfg!(windows) { "localbiseo-engine.exe" } else { "localbiseo-engine" };
    let mut candidates: Vec<PathBuf> = Vec::new();
    if let Ok(res) = app.path().resource_dir() {
        candidates.push(res.join("engine").join(exe_name));
    }
    if let Ok(exe) = std::env::current_exe() {
```

```
        if let Some(dir) = exe.parent() {
            candidates.push(dir.join(exe_name));
        }
    }
    if let Some(bin) = candidates.into_iter().find(|p| p.exists()) {
        return Command::new(bin);
    }
    // 개발 모드: 저장소의 engine 폴더에서 파이썬으로 실행
    let python = std::env::var("LOCALBISEO_PYTHON").unwrap_or_else(|_| {
        if cfg!(windows) { "python".into() } else { "python3".into() }
    });
    let engine_dir = PathBuf::from(env!("CARGO_MANIFEST_DIR")).join("../../engine");
    let mut cmd = Command::new(python);
    cmd.current_dir(engine_dir).args(["-m", "localbiseo"]);
    cmd
}

pub fn start(app: &AppHandle, state: SharedEngine) {
    let token = uuid::Uuid::new_v4().simple().to_string();
    let mut cmd = engine_command(app);
    // 배포판에 같이 넣은 ffmpeg · tesseract 위치 (tauri.release.conf.json의 resources)
    if let Ok(res) = app.path().resource_dir() {
        cmd.env("LOCALBISEO_RESOURCES", res.join("tools"));
    }
    cmd.args(["--port", "0", "--token", &token, "--watch-stdin"])
        .stdin(Stdio::piped())
        .stdout(Stdio::piped())
        .stderr(Stdio::inherit());
    #[cfg(windows)]
    {
        use std::os::windows::process::CommandExt;
        const CREATE_NO_WINDOW: u32 = 0x0800_0000;
        cmd.creation_flags(CREATE_NO_WINDOW);
    }
    let mut child = match cmd.spawn() {
        Ok(c) => c,
        Err(e) => {
            state.finish(Err(format!("엔진을 실행하지 못했어요: {e}")));
            return;
        }
    };
    let stdout = child.stdout.take();
    *state.child.lock().unwrap() = Some(child);

    let (tx, rx) = mpsc::channel::<u16>();
    if let Some(out) = stdout {
        std::thread::spawn(move || {
            let mut sent = false;
            for line in BufReader::new(out).lines().map_while(Result::ok) {
                if !sent {
                    if let Some(port) = line.strip_prefix("LOCALBISEO_READY ") {
                        if let Ok(p) = port.trim().parse() {
                            let _ = tx.send(p);
                            sent = true;
                        }
                    }
                }
                // 이후 출력은 그냥 흘려보낸다 (파이프가 차서 엔진이 멈추지 않도록).
            }
        });
    }
    std::thread::spawn(move || {
```

```
125         // 처음 실행할 때는 백신 검사 등으로 느릴 수 있어 넉넉히 기다린다.
126         let result = match rx.recv_timeout(Duration::from_secs(90)) {
127             Ok(port) => Ok(EngineInfo { url: format!("http://127.0.0.1:{port}"), token }),
128             Err(_) => Err("엔진이 시작되지 않았어요. 앱을 다시 실행해 주세요.".into()),
129         };
130         state.finish(result);
131     });
132 }
```

```rust
mod desktop;
mod engine;

use std::sync::Arc;
use std::time::Duration;

use tauri::menu::{Menu, MenuItem};
use tauri::tray::{MouseButton, MouseButtonState, TrayIconBuilder, TrayIconEvent};
use tauri::{AppHandle, Emitter, Manager, RunEvent, WindowEvent};
use tauri_plugin_global_shortcut::{Code, GlobalShortcutExt, Modifiers, Shortcut, ShortcutState};

use engine::{EngineInfo, EngineState, SharedEngine};

#[tauri::command]
async fn engine_info(state: tauri::State<'_, SharedEngine>) -> Result<EngineInfo, String> {
    let engine = state.inner().clone();
    tauri::async_runtime::spawn_blocking(move || engine.wait(Duration::from_secs(120)))
        .await
        .map_err(|e| e.to_string())?
}

#[tauri::command]
async fn capture_screen(app: AppHandle) -> Result<String, String> {
    tauri::async_runtime::spawn_blocking(move || desktop::capture_primary(&app))
        .await
        .map_err(|e| e.to_string())?
}

fn show_main(app: &AppHandle) {
    if let Some(w) = app.get_webview_window("main") {
        let _ = w.unminimize();
        let _ = w.show();
        let _ = w.set_focus();
    }
}

fn setup_shortcuts(app: &AppHandle) -> tauri::Result<()> {
    let palette = Shortcut::new(Some(Modifiers::CONTROL | Modifiers::SHIFT), Code::Space);
    let ocr = Shortcut::new(Some(Modifiers::CONTROL | Modifiers::SHIFT), Code::KeyO);
    app.plugin(
        tauri_plugin_global_shortcut::Builder::new()
            .with_handler(move |app, shortcut, event| {
                if event.state() != ShortcutState::Pressed {
                    return;
                }
                if shortcut == &palette {
                    show_main(app);
                    let _ = app.emit("open-palette", ());
                } else if shortcut == &ocr {
                    let app = app.clone();
                    std::thread::spawn(move || {
                        if let Ok(path) = desktop::capture_primary(&app) {
                            let _ = app.emit("screen-captured", path);
                        }
                    });
                }
            })
            .build(),
    )?;
    // 다른 프로그램이 이미 쓰는 단축키면 등록이 실패할 수 있다. 앱은 그대로 동작한다.
    let _ = app.global_shortcut().register(palette);
    let _ = app.global_shortcut().register(ocr);
```

```
    Ok(())
}

fn setup_tray(app: &AppHandle) -> tauri::Result<()> {
    let open = MenuItem::with_id(app, "open", "로컬비서 열기", true, None::<&str>)?;
    let capture = MenuItem::with_id(app, "capture", "화면 글자 읽기", true, None::<&str>)?;
    let quit = MenuItem::with_id(app, "quit", "끝내기", true, None::<&str>)?;
    let menu = Menu::with_items(app, &[&open, &capture, &quit])?;
    let mut tray = TrayIconBuilder::with_id("main")
        .tooltip("로컬비서")
        .menu(&menu)
        .show_menu_on_left_click(false)
        .on_menu_event(|app, event| match event.id().as_ref() {
            "open" => show_main(app),
            "capture" => {
                let app = app.clone();
                std::thread::spawn(move || {
                    if let Ok(path) = desktop::capture_primary(&app) {
                        let _ = app.emit("screen-captured", path);
                    }
                });
            }
            "quit" => {
                app.state::<SharedEngine>().kill();
                app.exit(0);
            }
            _ => {}
        })
        .on_tray_icon_event(|tray, event| {
            if let TrayIconEvent::Click { button: MouseButton::Left, button_state: MouseButtonState::Up, .. } = event {
                show_main(tray.app_handle());
            }
        });
    if let Some(icon) = app.default_window_icon() {
        tray = tray.icon(icon.clone());
    }
    tray.build(app)?;
    Ok(())
}

#[cfg_attr(mobile, tauri::mobile_entry_point)]
pub fn run() {
    let engine: SharedEngine = Arc::new(EngineState::default());
    let app = tauri::Builder::default()
        .plugin(tauri_plugin_dialog::init())
        .plugin(tauri_plugin_opener::init())
        .manage(engine.clone())
        .invoke_handler(tauri::generate_handler![engine_info, capture_screen])
        .setup(move |app| {
            let handle = app.handle().clone();
            engine::start(&handle, engine.clone());
            setup_shortcuts(&handle)?;
            setup_tray(&handle)?;
            desktop::watch_clipboard(handle);
            Ok(())
        })
        .on_window_event(|window, event| {
            // 창을 닫아도 트레이에 남아 클립보드 기록과 단축키가 계속 동작한다.
            if let WindowEvent::CloseRequested { api, .. } = event {
                if window.label() == "main" {
                    api.prevent_close();
                    let _ = window.hide();
```

```
                }
            }
        })
        .build(tauri::generate_context!())
        .expect("로컬비서를 시작하지 못했어요");

    app.run(|app, event| {
        if let RunEvent::Exit = event {
            app.state::<SharedEngine>().kill();
        }
    });
}
```

```
1  // 릴리스 빌드에서 Windows 콘솔 창이 뜨지 않게 한다.
2  #![cfg_attr(not(debug_assertions), windows_subsystem = "windows")]
3
4  fn main() {
5      localbiseo_lib::run()
6  }
```

```tsx
import { Command, Loader2, Search as SearchIcon } from "lucide-react";
import { useCallback, useEffect, useMemo, useRef, useState, type ComponentType } from "react";
import { SystemProvider, useSystem } from "./components/system";
import { ToastProvider, useToast } from "./components/ui";
import { api, errorText, initEngine } from "./lib/api";
import { runFocusWatcher } from "./lib/focusTimer";
import { installDropListener, listen } from "./lib/platform";
import { ALL_ITEMS, NAV } from "./nav";
import Actions from "./pages/Actions";
import Clipboard from "./pages/Clipboard";
import Dedupe from "./pages/Dedupe";
import Focus from "./pages/Focus";
import Home from "./pages/Home";
import Mask from "./pages/Mask";
import Media from "./pages/Media";
import Notes from "./pages/Notes";
import Ocr from "./pages/Ocr";
import Papers from "./pages/Papers";
import Pdf from "./pages/Pdf";
import Rename from "./pages/Rename";
import Schedule from "./pages/Schedule";
import Search from "./pages/Search";
import Settings from "./pages/Settings";
import Transcribe from "./pages/Transcribe";
import Translate from "./pages/Translate";
import Writing from "./pages/Writing";

const PAGES: Record<string, ComponentType> = {
  home: Home, transcribe: Transcribe, mask: Mask, media: Media, actions: Actions, schedule: Schedule, notes: Notes,
  clipboard: Clipboard, ocr: Ocr, papers: Papers, translate: Translate, writing: Writing, rename: Rename, dedupe: Dedupe,
  pdf: Pdf, focus: Focus, search: Search, settings: Settings,
};

function pageFromHash(): string {
  const key = window.location.hash.replace(/^#\/?/, "").split("?")[0];
  return PAGES[key] ? key : "home";
}

export default function App() {
  const [ready, setReady] = useState(false);
  const [bootError, setBootError] = useState<string | null>(null);
  const [page, setPage] = useState(pageFromHash);

  useEffect(() => {
    initEngine()
      .then(() => setReady(true))
      .catch((e) => setBootError(errorText(e)));
    const onHash = () => setPage(pageFromHash());
    window.addEventListener("hashchange", onHash);
    return () => window.removeEventListener("hashchange", onHash);
  }, []);

  const navigate = useCallback((key: string) => {
    window.location.hash = `/${key}`;
    setPage(key);
  }, []);

  if (!ready) {
    return (
      <div className="splash">
        <div className="logo">로컬비서</div>
        {bootError ? <p className="error-text">{bootError}</p> : <p className="muted"><Loader2 size={16} className="spin" /> 엔진을 깨우는 중…</p>}
```

```
      </div>
    );
  }
  return (
    <ToastProvider>
      <SystemProvider navigate={navigate}>
        <Shell page={page} navigate={navigate} />
      </SystemProvider>
    </ToastProvider>
  );
}

function Shell({ page, navigate }: { page: string; navigate: (k: string) => void }) {
  const [palette, setPalette] = useState(false);
  const [dropping, setDropping] = useState(false);
  const { status } = useSystem();
  const toast = useToast();
  const Current = PAGES[page];

  useEffect(() => {
    runFocusWatcher((r) => toast(r.kind === "focus" ? `${r.minutes}분 집중 끝! 잠깐 쉬어요.` : "쉬는 시간 끝! 다시 집중해 볼까요?"));
  }, [toast]);

  useEffect(() => {
    const onKey = (e: KeyboardEvent) => {
      if ((e.ctrlKey || e.metaKey) && e.key.toLowerCase() === "k") {
        e.preventDefault();
        setPalette((v) => !v);
      }
      if (e.key === "Escape") setPalette(false);
    };
    window.addEventListener("keydown", onKey);
    installDropListener(setDropping);
    const offs = [
      listen("open-palette", () => setPalette(true)),
      listen<string>("screen-captured", (path) => {
        sessionStorage.setItem("lb-captured", path);
        navigate("ocr");
        window.dispatchEvent(new CustomEvent("lb-captured", { detail: path }));
      }),
      // 앱 셸이 복사 내용을 알려주면 기록한다 (설정에서 끌 수 있다).
      listen<string>("clipboard-text", (text) => {
        api.post("/clipboard", { text }).catch(() => {});
        window.dispatchEvent(new CustomEvent("lb-clipboard"));
      }),
    ];
    return () => {
      window.removeEventListener("keydown", onKey);
      offs.forEach((p) => p.then((off) => off()));
    };
  }, [navigate]);

  const lic = status?.license;
  return (
    <div className="shell">
      <aside className="sidebar">
        <div className="brand">
          <span className="brand-mark">비</span>
          <span>로컬비서</span>
        </div>
        <button className="palette-trigger" onClick={() => setPalette(true)}>
          <SearchIcon size={15} />
```

```
          <span>찾기 · 이동</span>
          <kbd>Ctrl K</kbd>
        </button>
        <nav>
          {NAV.map((g, gi) => (
            <div key={gi} className="nav-group">
              {g.title && <div className="nav-title">{g.title}</div>}
              {g.items.map((it) => (
                <button key={it.key} className={`nav-item ${page === it.key ? "active" : ""}`} onClick={() => navigate(it.key)}>
                  <it.icon size={17} />
                  <span>{it.label}</span>
                </button>
              ))}
            </div>
          ))}
        </nav>
        <div className="sidebar-foot">
          {lic?.status === "licensed" && <span className="muted small">정품 · {lic.holder ¦¦ lic.id}</span>}
          {lic?.status === "trial" && (
            <button className="linklike small" onClick={() => navigate("settings")}>
              체험판 {lic.days_left}일 남음
            </button>
          )}
          {lic?.status === "expired" && (
            <button className="linklike small danger" onClick={() => navigate("settings")}>
              체험 기간 끝 · 키 입력
            </button>
          )}
          <span className="muted small">v{status?.version}</span>
        </div>
      </aside>
      <main className="main">
        <Current key={page} />
      </main>
      {palette && <Palette onClose={() => setPalette(false)} navigate={navigate} />}
      {dropping && <div className="drop-overlay">여기에 놓으세요</div>}
    </div>
  );
}

type SearchHit = { kind: string; kind_label: string; ref: string; title: string; snippet: string };
const KIND_PAGE: Record<string, string> = { note: "notes", todo: "notes", clip: "clipboard", transcript: "transcribe", file: "search", contact:

function Palette({ onClose, navigate }: { onClose: () => void; navigate: (k: string) => void }) {
  const [q, setQ] = useState("");
  const [hits, setHits] = useState<SearchHit[]>([]);
  const [sel, setSel] = useState(0);
  const input = useRef<HTMLInputElement>(null);
  useEffect(() => input.current?.focus(), []);
  const pages = useMemo(() => {
    const s = q.trim().toLowerCase();
    return ALL_ITEMS.filter((it) => !s ¦¦ `${it.label} ${it.desc} ${it.keywords ?? ""}`.toLowerCase().includes(s));
  }, [q]);
  useEffect(() => {
    if (q.trim().length < 2) {
      setHits([]);
      return;
    }
    const t = setTimeout(() => {
      api
        .get<{ results: SearchHit[] }>("/search", { q, limit: 8 })
        .then((r) => setHits(r.results))
```

```
        .catch(() => setHits([]));
    }, 180);
    return () => clearTimeout(t);
  }, [q]);
  const rows = [
    ...pages.map((p) => ({ id: `p-${p.key}`, label: p.label, sub: p.desc, icon: p.icon, go: () => navigate(p.key) })),
    ...hits.map((h) => ({
      id: `h-${h.kind}-${h.ref}`,
      label: h.title,
      sub: `${h.kind_label} · ${h.snippet}`,
      icon: SearchIcon,
      go: () => {
        sessionStorage.setItem("lb-open", JSON.stringify(h));
        navigate(KIND_PAGE[h.kind] ?? "search");
      },
    })),
  ];
  const choose = (i: number) => {
    rows[i]?.go();
    onClose();
  };
  return (
    <div className="palette-backdrop" onMouseDown={onClose}>
      <div className="palette" onMouseDown={(e) => e.stopPropagation()} role="dialog" aria-label="찾기">
        <div className="palette-input">
          <Command size={16} />
          <input
            ref={input}
            value={q}
            placeholder="기능 이름이나 찾을 내용을 입력하세요"
            onChange={(e) => {
              setQ(e.target.value);
              setSel(0);
            }}
            onKeyDown={(e) => {
              if (e.key === "ArrowDown") (e.preventDefault(), setSel((s) => Math.min(rows.length - 1, s + 1)));
              if (e.key === "ArrowUp") (e.preventDefault(), setSel((s) => Math.max(0, s - 1)));
              if (e.key === "Enter") choose(sel);
            }}
          />
        </div>
        <div className="palette-list">
          {rows.length === 0 && <div className="muted pad">결과가 없어요</div>}
          {rows.map((r, i) => (
            <button key={r.id} className={`palette-row ${i === sel ? "sel" : ""}`} onMouseEnter={() => setSel(i)} onClick={() => choose(i)}>
              <r.icon size={16} />
              <span className="palette-label">{r.label}</span>
              <span className="muted truncate small">{r.sub}</span>
            </button>
          ))}
        </div>
      </div>
    </div>
  );
}
```

```tsx
import { createContext, useContext, type ReactNode } from "react";
import { api } from "../lib/api";
import { useData } from "../lib/hooks";
import { Notice } from "./ui";

export type ComponentStatus = { key: string; label: string; available: boolean; used_by: string; extra: string };
export type License = { status: "trial" | "licensed" | "expired"; days_left?: number; trial_ends?: string; holder?: string; id?: string };
export type SystemStatus = {
  version: string;
  os: string;
  data_dir: string;
  components: ComponentStatus[];
  ffmpeg: boolean;
  ocr_backend: string | null;
  korean_font: string | null;
  llm_model: string | null;
  license: License;
};

type Ctx = {
  status: SystemStatus | null;
  reload: () => void;
  navigate: (page: string) => void;
  has: (key: string) => boolean;
};

const SystemCtx = createContext<Ctx>({ status: null, reload: () => {}, navigate: () => {}, has: () => false });

export function SystemProvider({ children, navigate }: { children: ReactNode; navigate: (p: string) => void }) {
  const { data, reload } = useData(() => api.get<SystemStatus>("/system/status"));
  const has = (key: string) => (key === "ffmpeg" ? !!data?.ffmpeg : !!data?.components.find((c) => c.key === key)?.available);
  return <SystemCtx.Provider value={{ status: data, reload, navigate, has }}>{children}</SystemCtx.Provider>;
}

export const useSystem = () => useContext(SystemCtx);

/** 선택 구성요소가 없을 때 안내. */
export function NeedsComponent({ keys, children }: { keys: string[]; children?: ReactNode }) {
  const { status, has, navigate } = useSystem();
  if (!status) return null;
  const missing = keys.filter((k) => !has(k));
  if (!missing.length) return null;
  const labels = missing.map((k) => (k === "ffmpeg" ? "ffmpeg" : status.components.find((c) => c.key === k)?.label ?? k));
  return (
    <Notice tone="warn">
      <span>
        <b>{labels.join(", ")}</b> 구성요소가 아직 없어요. {children}
        <button className="linklike" onClick={() => navigate("settings")}>
          설정 &gt; 구성요소
        </button>
        에서 확인해 주세요.
      </span>
    </Notice>
  );
}
```

```tsx
import { CircleAlert, FolderOpen, Copy, Loader2, Plus, Upload, X } from "lucide-react";
import { createContext, useCallback, useContext, useEffect, useRef, useState, type ReactNode } from "react";
import { api, type Job } from "../lib/api";
import { baseName } from "../lib/format";
import { copyText, isTauri, onFileDrop, openPath, pickFiles, reveal, type Filter } from "../lib/platform";

export function Page({ title, desc, actions, children }: { title: string; desc?: string; actions?: ReactNode; children: ReactNode }) {
  return (
    <div className="page">
      <header className="page-head">
        <div>
          <h1>{title}</h1>
          {desc && <p className="muted">{desc}</p>}
        </div>
        {actions && <div className="row">{actions}</div>}
      </header>
      {children}
    </div>
  );
}

export function Card({ title, right, children, className = "" }: { title?: ReactNode; right?: ReactNode; children: ReactNode; className?: strin
  return (
    <section className={`card ${className}`}>
      {(title || right) && (
        <div className="card-head">
          {title && <h2>{title}</h2>}
          {right}
        </div>
      )}
      {children}
    </section>
  );
}

type BtnProps = React.ButtonHTMLAttributes<HTMLButtonElement> & { variant?: "primary" | "ghost" | "danger" | "default"; busy?: boolean; icon?:

export function Button({ variant = "default", busy, icon, small, children, className = "", disabled, ...rest }: BtnProps) {
  return (
    <button className={`btn btn-${variant} ${small ? "btn-sm" : ""} ${className}`} disabled={disabled || busy} {...rest}>
      {busy ? <Loader2 size={16} className="spin" /> : icon}
      {children && <span>{children}</span>}
    </button>
  );
}

export function Field({ label, hint, children, inline }: { label: string; hint?: ReactNode; children: ReactNode; inline?: boolean }) {
  return (
    <label className={`field ${inline ? "field-inline" : ""}`}>
      <span className="field-label">{label}</span>
      {children}
      {hint && <span className="field-hint">{hint}</span>}
    </label>
  );
}

export function Toggle({ checked, onChange, label }: { checked: boolean; onChange: (v: boolean) => void; label: string }) {
  return (
    <label className="toggle">
      <input type="checkbox" checked={checked} onChange={(e) => onChange(e.target.checked)} />
      <span className="toggle-track" aria-hidden />
      <span>{label}</span>
```

```
    </label>
  );
}

export function Tabs<K extends string>({ tabs, value, onChange }: { tabs: { key: K; label: string }[]; value: K; onChange: (k: K) => void }) {
  return (
    <div className="tabs" role="tablist">
      {tabs.map((t) => (
        <button key={t.key} role="tab" aria-selected={value === t.key} className={value === t.key ? "active" : ""} onClick={() => onChange(t.ke
          {t.label}
        </button>
      ))}
    </div>
  );
}

export function ErrorBox({ error }: { error: string | null | undefined }) {
  if (!error) return null;
  return (
    <div className="alert alert-error" role="alert">
      <CircleAlert size={16} />
      <span>{error}</span>
    </div>
  );
}

export function Notice({ children, tone = "info" }: { children: ReactNode; tone?: "info" | "warn" | "ok" }) {
  return <div className={`alert alert-${tone}`}>{children}</div>;
}

export function Empty({ icon, children }: { icon?: ReactNode; children: ReactNode }) {
  return (
    <div className="empty">
      {icon}
      <div>{children}</div>
    </div>
  );
}

export function Badge({ children, tone = "" }: { children: ReactNode; tone?: string }) {
  return <span className={`badge ${tone ? `badge-${tone}` : ""}`}>{children}</span>;
}

export function JobBar({ job, error, onCancel }: { job: Job | null; error?: string | null; onCancel?: () => void }) {
  if (error && (!job || job.status !== "running")) return <ErrorBox error={error} />;
  if (!job || job.status === "done") return null;
  if (job.status === "cancelled") return <Notice tone="warn">작업을 취소했어요.</Notice>;
  return (
    <div className="jobbar">
      <div className="jobbar-top">
        <span>
          <Loader2 size={14} className="spin" /> {job.message || job.title}
        </span>
        <span className="muted">{Math.round(job.progress * 100)}%</span>
        {onCancel && (
          <Button small variant="ghost" onClick={onCancel}>
            취소
          </Button>
        )}
      </div>
      <div className="progress">
        <div style={{ width: `${Math.max(3, job.progress * 100)}%` }} />
```

```
      </div>
    </div>
  );
}

/** 엔진이 만든 이미지 파일의 작은 미리보기. */
export function Thumb({ path, size = 220 }: { path: string; size?: number }) {
  const [src, setSrc] = useState<string | null>(null);
  useEffect(() => {
    let alive = true;
    api
      .post<{ data_url: string }>("/ocr/preview", { path, max_size: size * 2 })
      .then((r) => alive && setSrc(r.data_url))
      .catch(() => {});
    return () => {
      alive = false;
    };
  }, [path, size]);
  return src ? <img className="thumb" src={src} alt={baseName(path)} style={{ maxWidth: size, maxHeight: size }} /> : <div className="thumb thu
}

export function PathLink({ path, label }: { path: string; label?: string }) {
  return (
    <span className="pathlink">
      <button className="linklike" title={path} onClick={() => openPath(path)}>
        {label ?? baseName(path)}
      </button>
      <button className="iconbtn" title={isTauri ? "폴더에서 보기" : "경로 복사"} onClick={() => reveal(path)}>
        {isTauri ? <FolderOpen size={14} /> : <Copy size={14} />}
      </button>
    </span>
  );
}

/** 파일/폴더 고르기 + 끌어다 놓기. */
export function PathPicker({
  value,
  onChange,
  multiple,
  directory,
  filters,
  placeholder,
}: {
  value: string[];
  onChange: (v: string[]) => void;
  multiple?: boolean;
  directory?: boolean;
  filters?: Filter[];
  placeholder?: string;
}) {
  const ref = useRef(value);
  ref.current = value;
  useEffect(
    () =>
      onFileDrop((paths) => {
        const next = multiple ? Array.from(new Set([...ref.current, ...paths])) : paths.slice(0, 1);
        onChange(next);
      }),
    [multiple, onChange],
  );
  const add = async () => {
    const picked = await pickFiles({ multiple, directory, filters });
```

```
    if (!picked.length) return;
    onChange(multiple ? Array.from(new Set([...value, ...picked])) : picked.slice(0, 1));
  };
  return (
    <div className="picker">
      {value.length === 0 ? (
        <button className="dropzone" onClick={add}>
          <Upload size={22} />
          <span>{placeholder ?? (directory ? "폴더를 고르거나 끌어다 놓으세요" : "파일을 고르거나 여기로 끌어다 놓으세요")}</span>
        </button>
      ) : (
        <div className="picker-list">
          {value.map((p) => (
            <div key={p} className="picker-item" title={p}>
              <span className="truncate">{baseName(p)}</span>
              <span className="muted truncate small">{p}</span>
              <button className="iconbtn" onClick={() => onChange(value.filter((x) => x !== p))} aria-label="빼기">
                <X size={14} />
              </button>
            </div>
          ))}
          <div className="row">
            {(multiple || value.length === 0) && (
              <Button small icon={<Plus size={14} />} onClick={add}>
                더 넣기
              </Button>
            )}
            <Button small variant="ghost" onClick={() => onChange([])}>
              모두 빼기
            </Button>
          </div>
        </div>
      )}
    </div>
  );
}

export function CopyButton({ text, label = "복사" }: { text: string; label?: string }) {
  const [done, setDone] = useState(false);
  return (
    <Button
      small
      icon={<Copy size={14} />}
      onClick={async () => {
        await copyText(text);
        setDone(true);
        setTimeout(() => setDone(false), 1200);
      }}
    >
      {done ? "복사했어요" : label}
    </Button>
  );
}

// --- 토스트 ---
type Toast = { id: number; text: string; tone: "ok" | "error" };
const ToastCtx = createContext<(text: string, tone?: "ok" | "error") => void>(() => {});

export function ToastProvider({ children }: { children: ReactNode }) {
  const [items, setItems] = useState<Toast[]>([]);
  const push = useCallback((text: string, tone: "ok" | "error" = "ok") => {
    const id = Date.now() + Math.random();
```

```
    setItems((xs) => [...xs, { id, text, tone }]);
    setTimeout(() => setItems((xs) => xs.filter((x) => x.id !== id)), 3200);
  }, []);
  return (
    <ToastCtx.Provider value={push}>
      {children}
      <div className="toasts" aria-live="polite">
        {items.map((t) => (
          <div key={t.id} className={`toast toast-${t.tone}`}>
            {t.text}
          </div>
        ))}
      </div>
    </ToastCtx.Provider>
  );
}

export const useToast = () => useContext(ToastCtx);
```

```ts
import { engineInfo, isTauri } from "./platform";

let base = (import.meta.env.VITE_ENGINE_URL as string | undefined) ?? "http://127.0.0.1:8765";
let token = (import.meta.env.VITE_ENGINE_TOKEN as string | undefined) ?? "";

export class ApiError extends Error {
  status: number;
  data: Record<string, unknown>;
  constructor(message: string, status: number, data: Record<string, unknown> = {}) {
    super(message);
    this.status = status;
    this.data = data;
  }
  get component(): string | undefined {
    return this.data.component as string | undefined;
  }
}

export async function initEngine(): Promise<void> {
  if (isTauri) {
    const info = await engineInfo();
    base = info.url;
    token = info.token;
  }
  // 엔진이 막 떠서 아직 준비 중일 수 있으니 잠깐 기다린다.
  for (let i = 0; i < 100; i++) {
    try {
      const r = await fetch(`${base}/health`);
      if (r.ok) return;
    } catch {
      /* 아직 안 떴음 */
    }
    await new Promise((res) => setTimeout(res, 200));
  }
  throw new Error("엔진에 연결하지 못했어요. 앱을 다시 실행해 주세요.");
}

async function request<T>(method: string, path: string, body?: unknown): Promise<T> {
  let res: Response;
  try {
    res = await fetch(`${base}${path}`, {
      method,
      headers: { "Content-Type": "application/json", "X-Biseo-Token": token },
      body: body === undefined ? undefined : JSON.stringify(body),
    });
  } catch {
    throw new ApiError("엔진과 연결이 끊겼어요.", 0);
  }
  const text = await res.text();
  const data = text ? JSON.parse(text) : null;
  if (!res.ok) {
    throw new ApiError((data && data.error) || `오류 (${res.status})`, res.status, data || {});
  }
  return data as T;
}

export const api = {
  get: <T,>(path: string, params?: Record<string, string | number | undefined>) => {
    const q = params
      ? "?" +
        Object.entries(params)
          .filter(([, v]) => v !== undefined && v !== "")
```

```
          .map(([k, v]) => `${encodeURIComponent(k)}=${encodeURIComponent(String(v))}`)
          .join("&")
      : "";
    return request<T>("GET", path + q);
  },
  post: <T,>(path: string, body: unknown = {}) => request<T>("POST", path, body),
  put: <T,>(path: string, body: unknown = {}) => request<T>("PUT", path, body),
  patch: <T,>(path: string, body: unknown = {}) => request<T>("PATCH", path, body),
  del: <T,>(path: string) => request<T>("DELETE", path),
};

export type Job<R = any> = {
  id: string;
  kind: string;
  title: string;
  status: "queued" | "running" | "done" | "error" | "cancelled";
  progress: number;
  message: string;
  result: R | null;
  error: string | null;
};

export type JobRef = { job_id: string; job: Job };

export function errorText(e: unknown): string {
  if (e instanceof Error) return e.message;
  return String(e);
}
```

```ts
// 집중 타이머 상태. 페이지를 옮겨도 계속 돌도록 화면 밖(모듈)에서 관리한다.
import { api } from "./api";

export type Running = { id: number; kind: "focus" | "break"; label: string; endsAt: number; minutes: number };
const KEY = "lb-focus-running";
const listeners = new Set<() => void>();

function read(): Running | null {
  try {
    return JSON.parse(localStorage.getItem(KEY) || "null");
  } catch {
    return null;
  }
}

function write(r: Running | null) {
  try {
    if (r) localStorage.setItem(KEY, JSON.stringify(r));
    else localStorage.removeItem(KEY);
  } catch {
    /* 저장소를 못 쓰면 이번 실행 동안만 유지 */
  }
  memo = r;
  listeners.forEach((l) => l());
}

let memo: Running | null = read();

export const focusTimer = {
  get: () => memo,
  subscribe(l: () => void) {
    listeners.add(l);
    return () => listeners.delete(l);
  },
  async start(kind: "focus" | "break", minutes: number, label: string) {
    const s = await api.post<{ id: number }>("/focus/start", { kind, planned_minutes: minutes, label });
    write({ id: s.id, kind, label, minutes, endsAt: Date.now() + minutes * 60_000 });
  },
  async stop(completed: boolean) {
    const r = memo;
    write(null);
    if (r) await api.post(`/focus/${r.id}/stop`, { completed }).catch(() => {});
  },
};

function chime() {
  try {
    const ctx = new AudioContext();
    [0, 0.25, 0.5].forEach((t, i) => {
      const o = ctx.createOscillator();
      const g = ctx.createGain();
      o.frequency.value = [660, 880, 990][i];
      g.gain.setValueAtTime(0.0001, ctx.currentTime + t);
      g.gain.exponentialRampToValueAtTime(0.25, ctx.currentTime + t + 0.02);
      g.gain.exponentialRampToValueAtTime(0.0001, ctx.currentTime + t + 0.22);
      o.connect(g).connect(ctx.destination);
      o.start(ctx.currentTime + t);
      o.stop(ctx.currentTime + t + 0.25);
    });
  } catch {
    /* 소리를 못 내도 괜찮다 */
  }
```

```
63 }
64
65 let started = false;
66 export function runFocusWatcher(onFinish: (r: Running) => void) {
67   if (started) return;
68   started = true;
69   window.setInterval(() => {
70     const r = memo;
71     if (r && Date.now() >= r.endsAt) {
72       focusTimer.stop(true);
73       chime();
74       onFinish(r);
75     }
76     listeners.forEach((l) => l());
77   }, 1000);
78 }
```

```ts
export function bytes(n: number): string {
  if (n < 1024) return `${n} B`;
  const units = ["KB", "MB", "GB", "TB"];
  let v = n / 1024;
  let i = 0;
  while (v >= 1024 && i < units.length - 1) {
    v /= 1024;
    i++;
  }
  return `${v.toFixed(v < 10 ? 1 : 0)} ${units[i]}`;
}

export function won(n: number | null | undefined): string {
  return n == null ? "-" : `${n.toLocaleString("ko-KR")}원`;
}

export function clock(sec: number): string {
  const s = Math.max(0, Math.floor(sec));
  const h = Math.floor(s / 3600);
  const m = Math.floor((s % 3600) / 60);
  const r = s % 60;
  const mm = String(m).padStart(2, "0");
  const ss = String(r).padStart(2, "0");
  return h ? `${h}:${mm}:${ss}` : `${mm}:${ss}`;
}

export function dateTime(iso: string | null | undefined): string {
  if (!iso) return "";
  const d = new Date(iso);
  if (Number.isNaN(d.getTime())) return iso;
  const sameYear = d.getFullYear() === new Date().getFullYear();
  return d.toLocaleString("ko-KR", {
    year: sameYear ? undefined : "numeric",
    month: "short",
    day: "numeric",
    weekday: "short",
    hour: iso.length > 10 ? "2-digit" : undefined,
    minute: iso.length > 10 ? "2-digit" : undefined,
  });
}

export function baseName(path: string): string {
  return path.split(/[\\/]/).pop() || path;
}

export function dueLabel(due: string | null | undefined): { text: string; tone: "late" | "today" | "soon" | "" } {
  if (!due) return { text: "", tone: "" };
  const today = new Date();
  today.setHours(0, 0, 0, 0);
  const d = new Date(due.slice(0, 10) + "T00:00:00");
  const diff = Math.round((d.getTime() - today.getTime()) / 86400000);
  if (diff < 0) return { text: `${-diff}일 지남`, tone: "late" };
  if (diff === 0) return { text: "오늘", tone: "today" };
  if (diff === 1) return { text: "내일", tone: "soon" };
  if (diff < 7) return { text: `${diff}일 뒤`, tone: "soon" };
  return { text: dateTime(due), tone: "" };
}

/** 브라우저 현지 시간 기준 YYYY-MM-DD (toISOString은 UTC라 아침에 하루가 밀린다). */
export function localDate(d: Date = new Date()): string {
  const p = (n: number) => String(n).padStart(2, "0");
  return `${d.getFullYear()}-${p(d.getMonth() + 1)}-${p(d.getDate())}`;
```

```
63  }
```

```ts
import { useCallback, useEffect, useRef, useState } from "react";
import { api, errorText, type Job, type JobRef } from "./api";

/** 긴 작업(job)을 시작하고 끝날 때까지 상태를 따라간다. */
export function useJob<R = any>() {
  const [job, setJob] = useState<Job<R> | null>(null);
  const [error, setError] = useState<string | null>(null);
  const timer = useRef<number | null>(null);
  const alive = useRef(true);

  useEffect(() => {
    alive.current = true;
    return () => {
      alive.current = false;
      if (timer.current) window.clearTimeout(timer.current);
    };
  }, []);

  const poll = useCallback((id: string, resolve: (j: Job<R>) => void) => {
    const tick = async () => {
      try {
        const j = await api.get<Job<R>>(`/jobs/${id}`);
        if (!alive.current) return;
        setJob(j);
        if (j.status === "done" || j.status === "error" || j.status === "cancelled") {
          if (j.status === "error") setError(j.error);
          resolve(j);
          return;
        }
      } catch (e) {
        setError(errorText(e));
        return;
      }
      timer.current = window.setTimeout(tick, 400);
    };
    tick();
  }, []);

  const start = useCallback(
    (run: () => Promise<JobRef>) =>
      new Promise<Job<R> | null>(async (resolve) => {
        setError(null);
        setJob(null);
        try {
          const ref = await run();
          setJob(ref.job as Job<R>);
          poll(ref.job_id, resolve);
        } catch (e) {
          setError(errorText(e));
          resolve(null);
        }
      }),
    [poll],
  );

  const cancel = useCallback(async () => {
    if (job) await api.post(`/jobs/${job.id}/cancel`);
  }, [job]);

  const running = !!job && (job.status === "queued" || job.status === "running");
  return { job, error, running, start, cancel, reset: () => (setJob(null), setError(null)) };
}
```

```
        ctx.db.execute(f"UPDATE transcripts SET {', '.join(f'{k}=?' for k in fields)} WHERE id=?", (*fields.values(), tid))
        _index(ctx.db, tid)
    return get_transcript(ctx.db, tid)


@router.post("/transcripts/{tid}/summarize")
def resummarize(tid: int, ctx: Ctx):
    t = get_transcript(ctx.db, tid)
    res = summarize(export.full_text(t["segments"]))
    ctx.db.execute("UPDATE transcripts SET summary=? WHERE id=?", (json.dumps(res["points"], ensure_ascii=False), tid))
    _index(ctx.db, tid)
    return res


@router.delete("/transcripts/{tid}")
def delete(tid: int, ctx: Ctx):
    ctx.db.execute("DELETE FROM transcripts WHERE id=?", (tid,))
    search.remove(ctx.db, "transcript", tid)
    return {"ok": True}


def _masked_copy(t: dict) -> dict:
    t = dict(t)
    t["segments"] = [{**s, "text": mask_text(s["text"])[0]} for s in t["segments"]]
    t["summary"] = [mask_text(p)[0] for p in t["summary"]]
    t["title"] = mask_text(t["title"])[0]
    return t


@router.post("/transcripts/{tid}/export")
def export_api(tid: int, body: ExportIn, ctx: Ctx):
    if body.format not in export.FORMATS:
        raise AppError(f"형식은 {', '.join(export.FORMATS)} 중 하나예요.")
    t = get_transcript(ctx.db, tid)
    actions = extract_actions("\n".join(s["text"] for s in t["segments"])) if body.include_actions else None
    if body.masked:
        t = _masked_copy(t)
        if actions:
            actions = [{**a, "text": mask_text(a["text"])[0]} for a in actions]
    if body.out_path:
        path = Path(body.out_path)
    else:
        safe = "".join(c for c in (t["title"] or "회의록") if c not in '\\/:*?"<>|').strip() or "회의록"
        path = sub_dir("exports") / f"{safe}{'_마스킹' if body.masked else ''}.{body.format}"
    path.parent.mkdir(parents=True, exist_ok=True)
    if body.format == "docx":
        export.to_docx(t, path, actions)
    else:
        content = {
            "txt": lambda: export.to_txt(t, body.timestamps),
            "md": lambda: export.to_md(t, actions),
            "srt": lambda: export.to_srt(t["segments"], t["speakers"]),
            "vtt": lambda: export.to_vtt(t["segments"], t["speakers"]),
        }[body.format]()
        path.write_text(content, encoding="utf-8")
    return {"path": str(path)}
```

```python
"""회의록 내보내기: txt, md, srt, vtt, docx."""
from __future__ import annotations

from pathlib import Path

FORMATS = ("txt", "md", "srt", "vtt", "docx")


def _ts(sec: float, sep: str = ",") -> str:
    ms = int(round(sec * 1000))
    h, ms = divmod(ms, 3_600_000)
    m, ms = divmod(ms, 60_000)
    s, ms = divmod(ms, 1000)
    return f"{h:02d}:{m:02d}:{s:02d}{sep}{ms:03d}"


def _clock(sec: float) -> str:
    sec = int(sec)
    h, rem = divmod(sec, 3600)
    m, s = divmod(rem, 60)
    return f"{h:d}:{m:02d}:{s:02d}" if h else f"{m:02d}:{s:02d}"


def speaker_name(seg: dict, speakers: dict) -> str:
    sp = seg.get("speaker")
    if sp is None:
        return ""
    return speakers.get(str(sp)) or f"화자{int(sp) + 1}"


def paragraphs(segments: list[dict], speakers: dict, gap: float = 1.5) -> list[dict]:
    """화자가 바뀌거나 오래 쉬면 문단을 나눈다."""
    out: list[dict] = []
    for seg in segments:
        name = speaker_name(seg, speakers)
        if out and out[-1]["speaker"] == name and seg["start"] - out[-1]["end"] < gap:
            out[-1]["text"] += " " + seg["text"].strip()
            out[-1]["end"] = seg["end"]
        else:
            out.append({"speaker": name, "start": seg["start"], "end": seg["end"], "text": seg["text"].strip()})
    return out


def full_text(segments: list[dict]) -> str:
    return " ".join(s["text"].strip() for s in segments)


def to_txt(t: dict, timestamps: bool = True) -> str:
    lines = []
    for p in paragraphs(t["segments"], t.get("speakers", {})):
        head = []
        if timestamps:
            head.append(f"[{_clock(p['start'])}]")
        if p["speaker"]:
            head.append(f"{p['speaker']}:")
        lines.append((" ".join(head) + " " if head else "") + p["text"])
    return "\n\n".join(lines) + "\n"


def to_srt(segments: list[dict], speakers: dict | None = None) -> str:
    blocks = []
    for i, s in enumerate(segments, 1):
```

```
        name = speaker_name(s, speakers or {})
        text = f"{name}: {s['text'].strip()}" if name else s["text"].strip()
        blocks.append(f"{i}\n{_ts(s['start'])} --> {_ts(s['end'])}\n{text}\n")
    return "\n".join(blocks)


def to_vtt(segments: list[dict], speakers: dict | None = None) -> str:
    blocks = ["WEBVTT\n"]
    for s in segments:
        name = speaker_name(s, speakers or {})
        text = f"<v {name}>{s['text'].strip()}" if name else s["text"].strip()
        blocks.append(f"{_ts(s['start'], '.')} --> {_ts(s['end'], '.')}\n{text}\n")
    return "\n".join(blocks)


def to_md(t: dict, actions: list[dict] | None = None) -> str:
    out = [f"# {t.get('title') or '회의록'}", ""]
    if t.get("created_at"):
        out += [f"- 작성: {t['created_at'][:16].replace('T', ' ')}", ""]
    summary = t.get("summary") or []
    if summary:
        out += ["## 요약", *[f"- {p}" for p in summary], ""]
    if actions:
        out.append("## 할 일")
        for a in actions:
            extra = " / ".join(x for x in (a.get("assignee"), a.get("due")) if x)
            out.append(f"- [ ] {a['text']}" + (f" ({extra})" if extra else ""))
        out.append("")
    out += ["## 전체 내용", ""]
    for p in paragraphs(t["segments"], t.get("speakers", {})):
        who = f"**{p['speaker']}** " if p["speaker"] else ""
        out += [f"{who}`{_clock(p['start'])}` {p['text']}", ""]
    return "\n".join(out)


def to_docx(t: dict, path: Path, actions: list[dict] | None = None) -> None:
    import docx
    from docx.shared import Pt, RGBColor

    doc = docx.Document()
    style = doc.styles["Normal"]
    style.font.name = "맑은 고딕"
    style.font.size = Pt(10.5)
    doc.add_heading(t.get("title") or "회의록", level=1)
    if t.get("created_at"):
        doc.add_paragraph(f"작성: {t['created_at'][:16].replace('T', ' ')}")
    summary = t.get("summary") or []
    if summary:
        doc.add_heading("요약", level=2)
        for p in summary:
            doc.add_paragraph(p, style="List Bullet")
    if actions:
        doc.add_heading("할 일", level=2)
        for a in actions:
            extra = " / ".join(x for x in (a.get("assignee"), a.get("due")) if x)
            doc.add_paragraph(f" {a['text']}" + (f" ({extra})" if extra else ""))
    doc.add_heading("전체 내용", level=2)
    for p in paragraphs(t["segments"], t.get("speakers", {})):
        para = doc.add_paragraph()
        stamp = para.add_run(f"[{_clock(p['start'])}] ")
        stamp.font.color.rgb = RGBColor(0x88, 0x88, 0x88)
        if p["speaker"]:
```

```
125                 para.add_run(f"{p['speaker']}: ").bold = True
126             para.add_run(p["text"])
127         doc.save(str(path))
```

```python
"""음성 인식(faster-whisper)과 실험적 화자 분리(speechbrain)."""
from __future__ import annotations

import os
import threading
from pathlib import Path

from ...core import optional
from ...core.config import sub_dir
from ...core.jobs import JobContext

MODEL_SIZES = {
    "base": "빠름 (정확도 보통, 약 150MB)",
    "small": "균형 (권장, 약 500MB)",
    "medium": "정확함 (느림, 약 1.5GB)",
    "large-v3": "가장 정확함 (아주 느림, 약 3GB, 16GB RAM 이상)",
}

_lock = threading.Lock()
_models: dict[str, object] = {}


def whisper_dir() -> Path:
    return sub_dir("whisper", sub_dir("models"))


def installed_models() -> list[str]:
    root = whisper_dir()
    out = []
    for size in MODEL_SIZES:
        # huggingface_hub 캐시 폴더 이름 규칙
        if any(root.glob(f"models--Systran--faster-whisper-{size}/snapshots/*/model.bin")):
            out.append(size)
    return out


def _model(size: str):
    optional.require("stt")
    from faster_whisper import WhisperModel

    with _lock:
        if size not in _models:
            _models.clear()  # 메모리를 아끼려고 한 번에 하나만 올린다
            _models[size] = WhisperModel(
                size,
                device="auto",
                compute_type="int8",
                download_root=str(whisper_dir()),
                cpu_threads=max(1, (os.cpu_count() or 4) - 1),
            )
        return _models[size]


def download_model(job: JobContext, size: str) -> dict:
    job.progress(0.05, f"{size} 모델 내려받는 중… (처음 한 번만)")
    _model(size)
    return {"size": size, "installed": installed_models()}


def transcribe(job: JobContext, path: str, size: str = "small", language: str | None = "ko") -> dict:
    model = _model(size)
    job.progress(0.02, "음성 분석 시작")
```

```
    segments_iter, info = model.transcribe(
        path,
        language=language or None,
        vad_filter=True,
        beam_size=5,
        condition_on_previous_text=False,
    )
    duration = float(info.duration or 0) or 1.0
    segments = []
    for seg in segments_iter:
        segments.append({"start": round(seg.start, 2), "end": round(seg.end, 2), "text": seg.text.strip(), "speaker": None})
        job.progress(min(0.95, seg.end / duration), f"{int(seg.end // 60)}분 {int(seg.end % 60)}초 / {int(duration // 60)}분")
    return {"segments": segments, "duration": duration, "language": info.language}


def diarize(job: JobContext, path: str, segments: list[dict], max_speakers: int | None = None, threshold: float = 0.55) -> list[dict]:
    """구간별 목소리 임베딩을 묶어 화자 번호를 붙인다. (실험 기능)"""
    optional.require("diarize")
    import numpy as np
    import torch
    from faster_whisper.audio import decode_audio
    from speechbrain.inference.speaker import EncoderClassifier

    job.progress(0.96, "화자 구분 중")
    encoder = EncoderClassifier.from_hparams(
        source="speechbrain/spkrec-ecapa-voxceleb", savedir=str(sub_dir("ecapa", sub_dir("models")))
    )
    sr = 16000
    wave = decode_audio(path, sampling_rate=sr)
    embs: list = []
    for seg in segments:
        a, b = int(seg["start"] * sr), int(seg["end"] * sr)
        if b - a < sr // 2:
            embs.append(None)
            continue
        with torch.no_grad():
            e = encoder.encode_batch(torch.from_numpy(wave[a:b]).unsqueeze(0)).squeeze().numpy()
        embs.append(e / (np.linalg.norm(e) + 1e-9))
    centroids: list = []
    counts: list[int] = []
    labels: list[int | None] = []
    for e in embs:
        if e is None:
            labels.append(None)
            continue
        sims = [float(np.dot(e, c / (np.linalg.norm(c) + 1e-9))) for c in centroids]
        best = int(np.argmax(sims)) if sims else -1
        if best >= 0 and sims[best] >= threshold:
            centroids[best] = centroids[best] + e
            counts[best] += 1
            labels.append(best)
        else:
            centroids.append(e.copy())
            counts.append(1)
            labels.append(len(centroids) - 1)
    # 너무 많이 쪼개졌으면 가장 가까운 묶음끼리 합친다.
    mapping = list(range(len(centroids)))
    if max_speakers and len(centroids) > max_speakers:
        alive = list(range(len(centroids)))
        while len(alive) > max_speakers:
            best_pair, best_sim = None, -2.0
            for i in alive:
```

```
                for j in alive:
                    if i < j:
                        ci, cj = centroids[i], centroids[j]
                        s = float(np.dot(ci, cj) / (np.linalg.norm(ci) * np.linalg.norm(cj) + 1e-9))
                        if s > best_sim:
                            best_pair, best_sim = (i, j), s
            i, j = best_pair
            centroids[i] = centroids[i] + centroids[j]
            alive.remove(j)
            mapping = [i if m == j else m for m in mapping]
        order = {c: n for n, c in enumerate(sorted(set(mapping)))}
        mapping = [order[m] for m in mapping]
    # 앞 구간과 같은 화자로 채워 빈 칸을 메운다.
    prev = 0
    out = []
    for seg, lab in zip(segments, labels):
        sp = mapping[lab] if lab is not None else prev
        prev = sp
        out.append({**seg, "speaker": sp})
    return out
```

```python
"""요약: 로컬 AI 모델이 있으면 그걸 쓰고, 없으면 핵심 문장 추출."""
from __future__ import annotations

import math
from collections import Counter

from ...core import llm
from ...core.korean import content_words, split_sentences

KEY_HINTS = ("결정", "합의", "결론", "중요", "요약", "마감", "까지", "확정", "문제", "이슈", "목표", "다음", "예산", "일정")


def extractive(text: str, max_sentences: int = 5) -> list[str]:
    sentences = [s for _, _, s in split_sentences(text) if len(s) >= 8]
    if len(sentences) <= max_sentences:
        return sentences
    freq = Counter(w for s in sentences for w in content_words(s))
    if not freq:
        return sentences[:max_sentences]
    top = freq.most_common(1)[0][1]
    scores = []
    for i, s in enumerate(sentences):
        words = set(content_words(s))
        if not words:
            scores.append(0.0)
            continue
        score = sum(freq[w] / top for w in words) / math.sqrt(len(words) + 1)
        if any(h in s for h in KEY_HINTS):
            score *= 1.3
        if i < len(sentences) * 0.1:
            score *= 1.1
        scores.append(score)
    picked = sorted(sorted(range(len(sentences)), key=lambda i: scores[i], reverse=True)[:max_sentences])
    return [sentences[i] for i in picked]


def summarize(text: str, max_sentences: int = 5) -> dict:
    if llm.available() and len(text) > 200:
        prompt = (
            "다음은 회의/강의 녹취록이야. 핵심 내용을 한국어 글머리표 3~6개로 요약해. "
            "결정된 사항과 해야 할 일이 있으면 꼭 포함해. 녹취록에 없는 내용은 쓰지 마.\n\n" + text[:12000]
        )
        try:
            out = llm.complete(prompt, max_tokens=600)
            lines = [l.strip(" -•*\t") for l in out.splitlines() if l.strip(" -•*\t")]
            if lines:
                return {"method": "llm", "points": lines}
        except Exception:  # noqa: BLE001 - 모델 실패 시 규칙 기반으로 넘어간다
            pass
    return {"method": "extractive", "points": extractive(text, max_sentences)}
```

```python
"""오프라인 번역 (Argos Translate). 언어팩은 처음 한 번 내려받는다."""
from __future__ import annotations

from fastapi import APIRouter
from pydantic import BaseModel

from ..core import llm, optional
from ..core.context import Ctx, job_ref
from ..core.errors import AppError, FeatureUnavailable
from ..core.jobs import JobContext

router = APIRouter(prefix="/translate", tags=["번역"])

LANGS = {"ko": "한국어", "en": "영어", "ja": "일본어", "zh": "중국어"}


def _installed_pairs() -> set[tuple[str, str]]:
    if not optional.available("translate"):
        return set()
    from argostranslate import package

    return {(p.from_code, p.to_code) for p in package.get_installed_packages()}


def _route(src: str, dst: str, pairs: set[tuple[str, str]]) -> list[tuple[str, str]] | None:
    if (src, dst) in pairs:
        return [(src, dst)]
    # 한↔일 같은 조합은 영어를 거쳐 간다.
    if (src, "en") in pairs and ("en", dst) in pairs:
        return [(src, "en"), ("en", dst)]
    return None


def translate(text: str, src: str, dst: str) -> dict:
    if src == dst:
        return {"text": text, "engine": "none"}
    pairs = _installed_pairs()
    route = _route(src, dst, pairs)
    if route:
        from argostranslate import translate as at

        out = text
        for a, b in route:
            out = "\n".join(at.translate(line, a, b) if line.strip() else line for line in out.split("\n"))
        return {"text": out, "engine": "argos", "route": route}
    if llm.available():
        prompt = f"다음 글을 {LANGS.get(dst, dst)}로 자연스럽게 번역해. 번역문만 출력해.\n\n{text}"
        return {"text": llm.complete(prompt, max_tokens=1500), "engine": "llm"}
    if not optional.available("translate"):
        optional.require("translate")
    raise FeatureUnavailable(
        f"{LANGS.get(src, src)}→{LANGS.get(dst, dst)} 언어팩이 없어요. 번역 화면에서 언어팩을 내려받아 주세요.",
        component="translate_pack",
    )


def install_pair(job: JobContext, src: str, dst: str) -> dict:
    optional.require("translate")
    from argostranslate import package

    job.progress(0.05, "언어팩 목록 확인 중")
    package.update_package_index()
```

```
    available = package.get_available_packages()
    needed = [(src, dst)] if any(p.from_code == src and p.to_code == dst for p in available) else [(src, "en"), ("en", dst)]
    have = _installed_pairs()
    for i, (a, b) in enumerate(needed):
        if (a, b) in have:
            continue
        pkg = next((p for p in available if p.from_code == a and p.to_code == b), None)
        if pkg is None:
            raise AppError(f"{a}→{b} 언어팩을 찾지 못했어요.")
        job.progress(0.1 + 0.8 * i / len(needed), f"{LANGS.get(a, a)}→{LANGS.get(b, b)} 내려받는 중")
        package.install_from_path(pkg.download())
    return {"installed": sorted(f"{a}-{b}" for a, b in _installed_pairs())}


class TranslateIn(BaseModel):
    text: str
    source: str = "ko"
    target: str = "en"


class InstallIn(BaseModel):
    source: str
    target: str


@router.get("/status")
def status():
    pairs = _installed_pairs()
    return {
        "available": optional.available("translate"),
        "llm": llm.available(),
        "languages": [{"code": k, "label": v} for k, v in LANGS.items()],
        "installed": sorted(f"{a}-{b}" for a, b in pairs),
    }


@router.post("")
def translate_api(body: TranslateIn):
    if body.source not in LANGS or body.target not in LANGS:
        raise AppError("지원하지 않는 언어예요.")
    return translate(body.text, body.source, body.target)


@router.post("/install")
def install_api(body: InstallIn, ctx: Ctx):
    optional.require("translate")
    return job_ref(ctx.jobs.submit("translate_pack", f"언어팩 {body.source}→{body.target}", install_pair, body.source, body.target))
```

```python
"""맞춤법 교정(자주 틀리는 표현), 문체 바꾸기(해요체/합니다체/공문체), 이메일 틀."""
from __future__ import annotations

import re
from typing import Callable

from fastapi import APIRouter
from pydantic import BaseModel

from ..core import llm
from ..core.errors import AppError
from ..core.korean import (
    JONG_B, JONG_M, JONG_NONE, JONG_SS, JUNG_A, JUNG_AE, JUNG_E, JUNG_EO, JUNG_EU, JUNG_I, JUNG_O, JUNG_U, JUNG_WA, JUNG_WO,
    JUNG_YEO, compose, decompose, has_jong, is_hangul, jong_of, with_jong,
)

router = APIRouter(prefix="/writing", tags=["글다듬기"])

JONG_L = 8
JUNG_WAE, JUNG_OE = 10, 11

# (정규식, 바꿀 말, 이유)
SPELL_RULES: list[tuple[str, str | Callable[[re.Match], str], str]] = [
    (r"몇\s?일", "며칠", "'몇 일'이 아니라 '며칠'이 맞아요."),
    (r"어의\s?없", "어이없", "'어의없다'가 아니라 '어이없다'예요."),
    (r"금새", "금세", "'금시에'의 준말이라 '금세'예요."),
    (r"왠(만|일)", r"웬\1", "'웬만하다 · 웬일'이 맞아요. ('왠지'만 '왠')"),
    (r"왠(?=\s)", "웬", "'어찌 된'의 뜻이면 '웬'이에요."),
    (r"오랫만", "오랜만", "'오래간만'의 준말이라 '오랜만'이에요."),
    (r"(?<=[할갈볼줄올될쓸릴낼칠울])께(?=요|[\s.!?,]|$)", "게", "'-ㄹ게'가 맞아요. ('-ㄹ께' ×)"),
    (r"안\s?되(?=요|[.!?]|$)", "안 돼", "'되어'의 준말이라 '돼'예요."),
    (r"(?<![가-힣])되(?=요|서|야|[.!?]|$)", "돼", "'되어'의 준말이라 '돼'예요."),
    (r"(?<=[가-힣])되(?=요(?:[\s.!?,]|$)|서\s)", "돼", "'되어'의 준말이라 '돼'예요."),
    (r"됬", "됐", "'되었'의 준말이라 '됐'이에요."),
    (r"않\s?(하|되|돼)", r"안 \1", "뒤에 동사가 오면 '안'을 써요. ('않'은 '-지 않다')"),
    (r"일부로", "일부러", "'일부러'가 맞아요."),
    (r"설겆이", "설거지", "'설거지'가 표준어예요."),
    (r"희안", "희한", "'희한하다'가 맞아요."),
    (r"(깨끗|틈틈|일일|곰곰|번번|깊숙|따뜻|느긋)히", r"\1이", "'-이'로 적는 말이에요."),
    (r"(솔직|꼼꼼|열심|조용|급)이(?=[\s.,!?]|$)", r"\1히", "'-히'로 적는 말이에요."),
    (r"역활", "역할", "'역할()'이 맞아요."),
    (r"어떻해", "어떡해", "'어떻게 해'의 준말은 '어떡해'예요."),
    (r"뵈요", "봬요", "'뵈어요'의 준말이라 '봬요'예요."),
    (r"문안하", "무난하", "'무난하다()'가 맞아요."),
    (r"구지(?=\s)", "굳이", "'굳이'가 맞아요."),
    (r"요세(?=\s|[.,!?]|$)", "요새", "'요사이'의 준말이라 '요새'예요."),
    (r"제작년", "재작년", "'재작년()'이 맞아요."),
    (r"내노라", "내로라", "'내로라하다'가 맞아요."),
    (r"짜집기", "짜깁기", "'짜깁기'가 맞아요."),
    (r"얼만큼", "얼마큼", "'얼마만큼'의 준말은 '얼마큼'이에요."),
    (r"넓직", "널찍", "'널찍하다'가 맞아요."),
    (r"(?<=[있었했겠없])슴(?=[\s.!?,]|$)", "음", "'-음'으로 적어요. ('-슴' ×)"),
    (r"바래(요|서|야)", r"바라\1", "'바라다(희망)'의 활용은 '바라요'예요."),
    (r"바랬", "바랐", "'바라다'의 과거는 '바랐다'예요."),
    (r"\s+([.,!?])", r"\1", "문장부호 앞은 띄우지 않아요."),
    (r"[ \t]{2,}", " ", "빈칸이 두 번 이상 들어갔어요."),
]


SPACING_TAILS = ("수", "때", "것", "줄", "뿐", "만큼")


```

```
def _fix_spacing(text: str, changes: list[dict]) -> str:
    """'할수있다' → '할 수 있다'처럼 ㄹ 받침 뒤 의존명사를 띄운다."""
    out = []
    i = 0
    count = 0
    while i < len(text):
        ch = text[i]
        out.append(ch)
        if is_hangul(ch) and jong_of(ch) == JONG_L and i + 1 < len(text):
            for tail in SPACING_TAILS:
                if text.startswith(tail, i + 1):
                    after = text[i + 1 + len(tail) : i + 2 + len(tail)]
                    # '할수록', '갈때기'처럼 붙여 쓰는 말은 빼고, 뒤가 빈칸 · 조사 · 용언일 때만
                    if tail == "수" and text.startswith("수록", i + 1):
                        break
                    if after and after not in " 있없도는가를이에.,!?\n":
                        break
                    out.append(" ")
                    count += 1
                    break
        if ch == "수" and i + 1 < len(text) and text[i + 1] in "있없" and out[-2:-1] == [" "]:
            out.append(" ")
        i += 1
    if count:
        changes.append({"before": "할수", "after": "할 수", "reason": "의존명사(수 · 때 · 것 · 줄)는 띄어 써요.", "count": count})
    return "".join(out)


def spellcheck(text: str) -> dict:
    changes: list[dict] = []
    out = text
    for pattern, repl, reason in SPELL_RULES:
        rx = re.compile(pattern)
        found = list(rx.finditer(out))
        if not found:
            continue
        m0 = found[0]
        context = out[max(0, m0.start() - 2) : m0.end() + 2]
        before = context
        after = rx.sub(repl, context)
        out, n = rx.subn(repl, out)
        if before != after:
            changes.append({"before": before.strip() or "", "after": after.strip() or "", "reason": reason, "count": n})
    # ㄹ 받침 + 려고: 할려고 → 하려고
    rx = re.compile(r"([가-힣])려고")
    def _ryeo(m: re.Match) -> str:
        c = m.group(1)
        return (with_jong(c, JONG_NONE) if jong_of(c) == JONG_L and c not in "갈멀알열걸들살놀만" else c) + "려고"
    new = rx.sub(_ryeo, out)
    if new != out:
        changes.append({"before": "할려고", "after": "하려고", "reason": "'-려고' 앞에 ㄹ을 덧붙이지 않아요.", "count": 1})
        out = new
    out = _fix_spacing(out, changes)
    return {"text": out, "changes": changes}


# ---------------- 문체 ----------------
END_RE = re.compile(r"([가-힣]+)([.!?…~]*)(?=\s|$|[\"')\]])")

POLITE_TO_FORMAL_WORDS = {
    "아니에요": "아닙니다", "아녜요": "아닙니다", "거예요": "것입니다", "거에요": "것입니다", "해요": "합니다", "돼요": "됩니다",
    "아파요": "아픕니다", "바빠요": "바쁩니다", "나빠요": "나쁩니다", "기뻐요": "기쁩니다", "슬퍼요": "슬픕니다", "예뻐요": "예쁩니다",
```

```
    "써요": "씁니다", "꺼요": "끕니다", "떠요": "뜹니다", "커요": "큽니다", "몰라요": "모릅니다", "달라요": "다릅니다", "빨라요": "빠릅니다",
    "골라요": "고릅니다", "불러요": "부릅니다", "도와요": "돕습니다", "들어요": "듣습니다", "걸어요": "걷습니다", "지어요": "짓습니다",
    "나아요": "낫습니다", "그래요": "그렇습니다", "이래요": "이렇습니다", "저래요": "저렇습니다", "어때요": "어떻습니까", "줘요": "줍니다",
}


# 합니다체로 바꾸면 뜻이 달라지는 어미는 그대로 둔다.
KEEP_ENDINGS = ("지요", "죠", "네요", "데요", "군요", "거든요", "까요", "래요", "대요", "잖아요", "던데요", "는걸요")


def _polite_word_to_formal(word: str, question: bool) -> str | None:
    """해요체 어절 하나를 합니다체로. 바꿀 수 없으면 None."""
    tail = "니까" if question else "니다"
    for k, v in POLITE_TO_FORMAL_WORDS.items():
        if word.endswith(k):
            v = v[:-2] + "니까" if question and v.endswith("니다") else v
            return word[: -len(k)] + v
    for k, v in (("이에요", "입"), ("예요", "입"), ("에요", "입")):
        if word.endswith(k):
            return word[: -len(k)] + v + tail
    if word.endswith(("을게요", "을께요")):
        return word[:-3] + "겠습" + tail
    if word.endswith(("게요", "께요")) and len(word) > 2 and jong_of(word[-3]) == JONG_L:
        return word[:-3] + with_jong(word[-3], JONG_NONE) + "겠습" + tail
    if word.endswith("세요"):
        return word[:-2] + ("십니까" if question else "십시오")
    if word.endswith("나요") and len(word) > 2 and is_hangul(word[-3]):
        stem = word[:-2]
        if has_jong(stem[-1]):
            return stem + "습" + tail
        return stem[:-1] + with_jong(stem[-1], JONG_B) + tail
    if word.endswith(KEEP_ENDINGS):
        return None
    if not word.endswith("요") or len(word) < 2:
        return None
    stem = word[:-1]
    last = stem[-1]
    if not is_hangul(last):
        return None
    if stem.endswith(("어", "아")) and len(stem) >= 2 and is_hangul(stem[-2]):
        prev = stem[-2]
        j = jong_of(prev)
        if j == JONG_L:  # 알아요 → 압니다, 만들어요 → 만듭니다
            return stem[:-2] + with_jong(prev, JONG_B) + tail
        if j > 0:  # 있어요 → 있습니다
            return stem[:-1] + "습" + tail
        return None
    cho, jung, jong = decompose(last)
    if jong:
        return None
    if last == "워" and len(stem) >= 2:  # 어려워요 → 어렵습니다
        return stem[:-2] + with_jong(stem[-2], JONG_B) + "습" + tail
    new_jung = {JUNG_YEO: JUNG_I, JUNG_WA: JUNG_O, JUNG_WO: JUNG_U, JUNG_WAE: JUNG_OE}.get(jung, jung)
    if jung in (JUNG_A, JUNG_AE, JUNG_EO, JUNG_E, JUNG_YEO, JUNG_WA, JUNG_WO, JUNG_WAE, JUNG_I, JUNG_O, JUNG_U, JUNG_OE):
        return stem[:-1] + compose(cho, new_jung, JONG_B) + tail
    return None


def _formal_word_to_polite(word: str, question: bool) -> str | None:
    q = "?" if question else ""
    special = {"합니다": "해요", "합니까": "해요", "됩니다": "돼요", "됩니까": "돼요", "아닙니다": "아니에요", "아닙니까": "아니에요",
               "십시오": "세요", "십니다": "세요", "십니까": "세요", "것입니다": "거예요", "것입니까": "거예요",
```

```
                "겁니다": "거예요", "압니다": "알아요", "만듭니다": "만들어요", "엽니다": "열어요", "놉니다": "놀아요"}
    for k, v in special.items():
        if word.endswith(k):
            return word[: -len(k)] + v
    for k in ("입니다", "입니까"):
        if word.endswith(k):
            stem = word[: -len(k)]
            if not stem:
                return None
            return stem + ("이에요" if has_jong(stem[-1]) else "예요")
    for k in ("습니다", "습니까"):
        if word.endswith(k) and len(word) > 3:
            stem = word[: -len(k)]
            last = stem[-1]
            if not is_hangul(last):
                return None
            if stem.endswith(IRREGULAR_B):
                return stem[:-1] + with_jong(last, JONG_NONE) + "워요"
            if stem.endswith("돕"):
                return stem[:-1] + "도와요"
            _, jung, jong = decompose(last)
            if jong == JONG_SS:
                return stem + "어요"
            return stem + ("아요" if jung in (JUNG_A, JUNG_O, 2) else "어요")
    for k in ("니다", "니까"):
        if word.endswith(k) and len(word) > 2:
            stem = word[: -len(k)]
            last = stem[-1]
            if not is_hangul(last) or jong_of(last) != JONG_B:
                return None
            cho, jung, _ = decompose(last)
            base = stem[:-1]
            if jung == JUNG_O:
                return base + compose(cho, JUNG_WA) + "요"
            if jung == JUNG_U:
                return base + compose(cho, JUNG_WO) + "요"
            if jung == JUNG_OE:
                return base + compose(cho, JUNG_WAE) + "요"
            if jung == JUNG_I:
                return base + compose(cho, JUNG_YEO) + "요"
            if jung == JUNG_EU:
                prev_jung = decompose(base[-1])[1] if base and is_hangul(base[-1]) else JUNG_EO
                return base + compose(cho, JUNG_A if prev_jung in (JUNG_A, JUNG_O) else JUNG_EO) + "요"
            return base + compose(cho, jung) + "요"
    return None


IRREGULAR_B = ("어렵", "쉽", "고맙", "반갑", "덥", "춥", "가볍", "무겁", "귀엽", "즐겁", "아름답", "새롭", "어둡", "맵", "가깝", "아깝", "부럽", "그

def _formal_word_to_official(word: str) -> str | None:
    if word.endswith("습니다"):
        stem = word[:-3]
        if stem.endswith(IRREGULAR_B):  # 어렵습니다 → 어려움
            return stem[:-1] + with_jong(stem[-1], JONG_NONE) + "움"
        return stem + "음"
    if word.endswith("니다") and len(word) > 2 and jong_of(word[-3]) == JONG_B:
        return word[:-3] + with_jong(word[-3], JONG_M)
    if word.endswith("십시오"):
        return word[:-3] + "시기 바람"
    return None

```

```
249
250 def _convert(text: str, fn: Callable[[str, bool], str | None]) -> tuple[str, int]:
251     count = 0
252
253     def repl(m: re.Match) -> str:
254         nonlocal count
255         word, punct = m.group(1), m.group(2)
256         # 문장 끝 어절만 바꾼다 (문장부호가 있거나 줄 끝).
257         end = m.end()
258         at_end = bool(punct) or end == len(text) or text[end] == "\n"
259         if not at_end:
260             return m.group(0)
261         new = fn(word, "?" in punct)
262         if new and new != word:
263             count += 1
264             return new + punct
265         return m.group(0)
266
267     return END_RE.sub(repl, text), count
268
269
270 STYLES = {"formal": "합니다체", "polite": "해요체", "official": "공문체(~함/~임)"}
271
272
273 def change_tone(text: str, style: str) -> dict:
274     if style not in STYLES:
275         raise AppError("문체는 formal, polite, official 중 하나예요.")
276     if style == "polite":
277         out, n = _convert(text, _formal_word_to_polite)
278     else:
279         out, n = _convert(text, _polite_word_to_formal)
280         if style == "official":
281             out, n2 = _convert(out, lambda w, q: _formal_word_to_official(w))
282             n = max(n, n2)
283             # 개조식은 '~함', '~음' 뒤에 마침표를 찍지 않는다.
284             out = re.sub(r"([가-힣])\.(?=\s|$)", lambda m: m.group(1) if jong_of(m.group(1)) == JONG_M else m.group(0), out)
285     return {"text": out, "changed": n, "style": STYLES[style]}
286
287
288 def email(body: str, recipient: str = "", sender: str = "", company: str = "", subject: str = "", style: str = "formal") -> dict:
289     content = change_tone(body, "formal" if style == "formal" else "polite")["text"].strip()
290     to = f"{recipient}님" if recipient and not recipient.endswith("님") else recipient
291     me = f"{company} {sender}".strip()
292     lines = []
293     lines.append(f"안녕하세요, {to}." if to else "안녕하세요.")
294     if me:
295         lines.append(f"{me}입니다.")
296     lines += ["", content, ""]
297     lines.append("확인 부탁드립니다." if style == "formal" else "확인 부탁드려요.")
298     lines.append("감사합니다.")
299     if sender:
300         lines += ["", f"{me} 드림"]
301     return {"subject": subject or (content.splitlines()[0][:30] if content else ""), "text": "\n".join(lines)}
302
303
304 class TextIn(BaseModel):
305     text: str
306
307
308 class ToneIn(BaseModel):
309     text: str
310     style: str = "formal"
```

```
    use_ai: bool = False


class EmailIn(BaseModel):
    body: str
    recipient: str = ""
    sender: str = ""
    company: str = ""
    subject: str = ""
    style: str = "formal"


@router.post("/spell")
def spell_api(body: TextIn):
    return spellcheck(body.text)


@router.post("/tone")
def tone_api(body: ToneIn):
    if body.use_ai and llm.available():
        prompt = f"다음 글의 내용은 그대로 두고 {STYLES.get(body.style, '합니다체')}로 자연스럽게 고쳐 써. 고친 글만 출력해.\n\n{body.text}"
        return {"text": llm.complete(prompt, max_tokens=1500), "changed": -1, "style": STYLES.get(body.style), "engine": "llm"}
    return change_tone(body.text, body.style)


@router.post("/email")
def email_api(body: EmailIn):
    return email(body.body, body.recipient, body.sender, body.company, body.subject, body.style)
```

```
"""엔진을 실행 파일 하나로 묶어 Tauri 사이드카 자리에 둔다.

  python tools/build_engine.py                    # 현재 PC용
  python tools/build_engine.py --ffmpeg C:/ffmpeg/bin/ffmpeg.exe --tesseract "C:/Program Files/Tesseract-OCR"

결과:
  app/src-tauri/resources/engine/localbiseo-engine[.exe] (+ _internal 폴더)
  app/src-tauri/resources/tools/ffmpeg/…, tools/tesseract/… (넘긴 경우)

한 파일(onefile)로 묶으면 실행할 때마다 수백 MB를 임시 폴더에 풀어서 느리다.
그래서 폴더째(onedir) 앱 리소스로 넣는다.

미리 설치할 것: pip install "./engine[stt,ocr,rembg,face,ffmpeg]" pyinstaller
"""
from __future__ import annotations

import argparse
import shutil
import subprocess
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
ENGINE = ROOT / "engine"
TAURI = ROOT / "app" / "src-tauri"
FONT_CANDIDATES = [
    Path(r"C:\Windows\Fonts\NanumGothic.ttf"),
    Path("/usr/share/fonts/truetype/nanum/NanumGothic.ttf"),
    Path("/Library/Fonts/NanumGothic.ttf"),
]

# 동적으로 불러와서 PyInstaller가 놓치는 모듈
HIDDEN = [
    "uvicorn.logging", "uvicorn.loops.auto", "uvicorn.protocols.http.auto", "uvicorn.protocols.websockets.auto",
    "uvicorn.lifespan.on", "pypdfium2", "pypdfium2_raw", "olefile", "openpyxl", "docx",
]
COLLECT = ["faster_whisper", "ctranslate2", "rembg", "pytesseract", "cv2", "pypdfium2", "pypdfium2_raw", "docx", "imageio_ffmpeg"]


def target_triple() -> str:
    out = subprocess.run(["rustc", "-vV"], capture_output=True, text=True, check=True).stdout
    return next(line.split(": ", 1)[1] for line in out.splitlines() if line.startswith("host: "))


def installed(mod: str) -> bool:
    import importlib.util

    return importlib.util.find_spec(mod) is not None


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--target", help="Rust target triple (기본: 현재 PC)")
    ap.add_argument("--ffmpeg", help="함께 넣을 ffmpeg 실행 파일 (LGPL 빌드 권장)")
    ap.add_argument("--tesseract", help="함께 넣을 Tesseract 설치 폴더 (kor.traineddata 포함)")
    ap.add_argument("--font", help="워터마크용 한글 글꼴 (기본: 나눔고딕을 찾아서 넣음)")
    args = ap.parse_args()

    triple = args.target or target_triple()
    ext = ".exe" if "windows" in triple else ""
    work = ROOT / "engine" / "build"
    sep = ";" if sys.platform == "win32" else ":"
```

```
63
64      cmd = [
65          sys.executable, "-m", "PyInstaller", "--noconfirm", "--clean", "--onedir",
66          "--name", "localbiseo-engine",
67          "--distpath", str(work / "dist"), "--workpath", str(work / "work"), "--specpath", str(work),
68          "--add-data", f"{ENGINE / 'localbiseo' / 'core' / 'license_pubkey.pem'}{sep}localbiseo/core",
69      ]
70      font = Path(args.font) if args.font else next((f for f in FONT_CANDIDATES if f.exists()), None)
71      if font:
72          cmd += ["--add-data", f"{font}{sep}fonts"]
73      for h in HIDDEN:
74          cmd += ["--hidden-import", h]
75      for c in COLLECT:
76          if installed(c):
77              cmd += ["--collect-all", c]
78      if sys.platform != "win32":
79          cmd += ["--strip"]
80      cmd.append(str(ENGINE / "run_engine.py"))
81      print(" ".join(cmd))
82      subprocess.run(cmd, check=True, cwd=ENGINE)
83
84      engine_dst = TAURI / "resources" / "engine"
85      if engine_dst.exists():
86          shutil.rmtree(engine_dst)
87      shutil.copytree(work / "dist" / "localbiseo-engine", engine_dst)
88      print(f"엔진: {engine_dst} ({triple})")
89
90      tools = TAURI / "resources" / "tools"
91      tools.mkdir(parents=True, exist_ok=True)
92      (tools / "README.txt").write_text("로컬비서가 쓰는 외부 도구(ffmpeg, tesseract)가 이 폴더에 들어갑니다.\n", encoding="utf-8")
93      if args.ffmpeg:
94          (tools / "ffmpeg").mkdir(exist_ok=True)
95          shutil.copy2(args.ffmpeg, tools / "ffmpeg" / f"ffmpeg{ext}")
96          print("ffmpeg 포함")
97      if args.tesseract:
98          target = tools / "tesseract"
99          if target.exists():
100             shutil.rmtree(target)
101         shutil.copytree(args.tesseract, target, ignore=shutil.ignore_patterns("*.html", "doc", "uninstall*"))
102         print("tesseract 포함")
103
104
105 if __name__ == "__main__":
106     main()
```

```python
"""시연 영상 · 스크린샷용 예시 파일 만들기 (모두 가상의 인물 · 번호).

  python tools/demo/make_samples.py <폴더> [--audio]

--audio: edge-tts로 회의 녹음(meeting.mp3)도 만든다 (인터넷 필요).
"""
from __future__ import annotations

import asyncio
import sys
import zipfile
from pathlib import Path

from PIL import Image, ImageDraw, ImageFilter, ImageFont

FONT = "/usr/share/fonts/truetype/nanum/NanumGothic.ttf"
FONT_BOLD = "/usr/share/fonts/truetype/nanum/NanumGothicBold.ttf"

MEETING = (
    "오늘 회의 안건은 와디즈 펀딩 준비입니다. 목표 금액은 삼백만 원으로 정했습니다. "
    "김민수 대리가 금요일까지 상세페이지 시안을 공유하기로 했습니다. "
    "박지영 과장은 다음 주 월요일까지 촬영 장소 견적을 받아 주세요. "
    "다음 주 화요일 오후 두 시에 제품 촬영 일정을 잡았습니다. "
    "예산 문제는 다음 회의에서 다시 논의하겠습니다."
)

HWPX_SECTION = (
    '<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>'
    '<hs:sec xmlns:hs="http://www.hancom.co.kr/hwpml/2011/section" xmlns:hp="http://www.hancom.co.kr/hwpml/2011/paragraph">'
    '<hp:p id="1"><hp:run charPrIDRef="0"><hp:t>장학금 신청서</hp:t></hp:run></hp:p>'
    '<hp:p id="2"><hp:run charPrIDRef="0"><hp:t>신청인: 이서연</hp:t></hp:run></hp:p>'
    '<hp:p id="3"><hp:run charPrIDRef="0"><hp:t>연락처: 010-2468-1357</hp:t></hp:run></hp:p>'
    '<hp:p id="4"><hp:run charPrIDRef="0"><hp:t>주민등록번호: 030315-4123456</hp:t></hp:run></hp:p>'
    '<hp:p id="5"><hp:run charPrIDRef="0"><hp:t>이메일: seoyeon.lee@example.com</hp:t></hp:run></hp:p>'
    "</hs:sec>"
)


def product(path: Path, color: tuple[int, int, int], size=(1600, 1000)) -> None:
    w, h = size
    img = Image.new("RGB", size, (226, 221, 214))
    d = ImageDraw.Draw(img)
    d.rectangle((0, int(h * 0.64), w, h), fill=(205, 196, 184))
    sh = Image.new("L", size, 0)
    ImageDraw.Draw(sh).ellipse((w * 0.35, h * 0.61, w * 0.66, h * 0.69), fill=120)
    img.paste((150, 140, 128), mask=sh.filter(ImageFilter.GaussianBlur(18)))
    dark = tuple(int(c * 0.78) for c in color)
    light = tuple(min(255, int(c * 1.35) + 30) for c in color)
    d.rounded_rectangle((w * 0.375, h * 0.3, w * 0.6125, h * 0.66), radius=40, fill=color)
    d.ellipse((w * 0.375, h * 0.27, w * 0.6125, h * 0.33), fill=dark)
    d.ellipse((w * 0.3875, h * 0.28, w * 0.6, h * 0.322), fill=(30, 40, 60))
    d.ellipse((w * 0.6, h * 0.38, w * 0.68, h * 0.56), outline=color, width=34)
    d.rounded_rectangle((w * 0.4, h * 0.34, w * 0.419, h * 0.62), radius=14, fill=light)
    img.save(path, quality=92)


def text_image(path: Path, lines: list[str], size=(900, 300), title=False) -> None:
    img = Image.new("RGB", size, "white")
    d = ImageDraw.Draw(img)
    for i, line in enumerate(lines):
        f = ImageFont.truetype(FONT_BOLD if title and i == 0 else FONT, 34)
        d.text((30, 30 + i * 80), line, font=f, fill="black")
```

```
    img.save(path)


def main() -> None:
    out = Path(sys.argv[1])
    out.mkdir(parents=True, exist_ok=True)
    import docx

    doc = docx.Document()
    doc.add_heading("고객 상담 기록", 1)
    for line in ["상담일: 2026-10-20", "성명: 홍길동", "연락처: 010-1234-5678", "주민등록번호: 900101-1234567",
                 "주소: 서울특별시 강남구 테헤란로 123, 101동 1001호", "환불 계좌: 국민은행 123456-01-234567"]:
        doc.add_paragraph(line)
    doc.save(out / "고객상담기록.docx")

    with zipfile.ZipFile(out / "장학금신청서.hwpx", "w") as z:
        z.writestr(zipfile.ZipInfo("mimetype"), "application/hwp+zip")
        z.writestr("Contents/section0.xml", HWPX_SECTION, compress_type=zipfile.ZIP_DEFLATED)

    from fpdf import FPDF

    pdf = FPDF()
    pdf.add_font("n", fname=FONT)
    pdf.set_font("n", size=13)
    pdf.add_page()
    for line in ["보험금 청구서", "피보험자: 김철수", "주민등록번호: 850505-2345678", "연락처 010-9876-5432", "이메일 chulsoo.kim@example.com"]:
        pdf.cell(0, 10, line, new_x="LMARGIN", new_y="NEXT")
    pdf.output(str(out / "청구서.pdf"))

    text_image(out / "택배송장.png", ["택배 송장", "받는 분 연락처 010-5555-7777", "주소 부산광역시 해운대구 센텀로 45"], title=True)

    products = out / "상품사진"
    products.mkdir(exist_ok=True)
    product(products / "머그컵_블루.jpg", (47, 107, 237))
    product(products / "머그컵_코랄.jpg", (240, 112, 96))
    product(products / "머그컵_그린.jpg", (46, 158, 110), size=(1400, 1050))

    (out / "회의메모.txt").write_text(MEETING, encoding="utf-8")
    if "--audio" in sys.argv:
        import edge_tts

        asyncio.run(edge_tts.Communicate(MEETING, "ko-KR-SunHiNeural", rate="-5%").save(str(out / "회의녹음.mp3")))
    print(f"예시 파일: {out}")


if __name__ == "__main__":
    main()
```

```python
"""라이선스 키 발급 도구 (판매자 전용, 앱에 포함하지 않는다).

  # 1) 서명 키 만들기 (한 번만). 개인키는 절대 저장소에 올리지 말 것.
  python tools/license_gen.py keygen --private ~/.localbiseo-signing/private.pem

  # 2) 키 발급 (와디즈 서포터 1명당 1개)
  python tools/license_gen.py issue --private ~/.localbiseo-signing/private.pem --id WADIZ-0001 --name 홍길동

  # 3) CSV로 한꺼번에 발급 (열: id,name) → 키가 추가된 CSV 출력
  python tools/license_gen.py batch --private ~/.localbiseo-signing/private.pem backers.csv out.csv
"""
from __future__ import annotations

import argparse
import csv
import sys
from datetime import date
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "engine"))

from cryptography.hazmat.primitives import serialization  # noqa: E402
from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey  # noqa: E402

from localbiseo.core.license import sign  # noqa: E402

PUBKEY_OUT = Path(__file__).resolve().parent.parent / "engine" / "localbiseo" / "core" / "license_pubkey.pem"


def load_private(path: str) -> Ed25519PrivateKey:
    return serialization.load_pem_private_key(Path(path).expanduser().read_bytes(), password=None)


def cmd_keygen(args) -> None:
    priv_path = Path(args.private).expanduser()
    if priv_path.exists() and not args.force:
        sys.exit(f"이미 있어요: {priv_path} (덮어쓰려면 --force)")
    priv_path.parent.mkdir(parents=True, exist_ok=True)
    key = Ed25519PrivateKey.generate()
    priv_path.write_bytes(
        key.private_bytes(serialization.Encoding.PEM, serialization.PrivateFormat.PKCS8, serialization.NoEncryption())
    )
    priv_path.chmod(0o600)
    PUBKEY_OUT.write_bytes(
        key.public_key().public_bytes(serialization.Encoding.PEM, serialization.PublicFormat.SubjectPublicKeyInfo)
    )
    print(f"개인키: {priv_path}\n공개키(앱에 포함): {PUBKEY_OUT}")


def make_payload(lic_id: str, name: str, edition: str, exp: str | None) -> dict:
    payload = {"id": lic_id, "name": name, "ed": edition, "iat": date.today().isoformat()}
    if exp:
        payload["exp"] = exp
    return payload


def cmd_issue(args) -> None:
    print(sign(make_payload(args.id, args.name, args.edition, args.exp), load_private(args.private)))


def cmd_batch(args) -> None:
    key = load_private(args.private)
```

```
    with open(args.input, newline="", encoding="utf-8-sig") as f, open(args.output, "w", newline="", encoding="utf-8-sig") as out:
        reader = csv.DictReader(f)
        writer = csv.DictWriter(out, fieldnames=[*(reader.fieldnames or []), "license_key"])
        writer.writeheader()
        for row in reader:
            row["license_key"] = sign(make_payload(row["id"], row.get("name", ""), args.edition, args.exp), key)
            writer.writerow(row)
    print(f"완료: {args.output}")


def main() -> None:
    ap = argparse.ArgumentParser(description="로컬비서 라이선스 키 발급")
    sub = ap.add_subparsers(required=True)
    k = sub.add_parser("keygen")
    k.add_argument("--private", required=True)
    k.add_argument("--force", action="store_true")
    k.set_defaults(fn=cmd_keygen)
    for name, fn in (("issue", cmd_issue), ("batch", cmd_batch)):
        p = sub.add_parser(name)
        p.add_argument("--private", required=True)
        p.add_argument("--edition", default="pro")
        p.add_argument("--exp", help="만료일 YYYY-MM-DD (없으면 평생)")
        if name == "issue":
            p.add_argument("--id", required=True)
            p.add_argument("--name", default="")
        else:
            p.add_argument("input")
            p.add_argument("output")
        p.set_defaults(fn=fn)
    args = ap.parse_args()
    args.fn(args)


if __name__ == "__main__":
    main()
```

```
"""설치 파일에 넣을 오픈소스 고지문(THIRD_PARTY_NOTICES.txt)을 만든다.

  python tools/notices.py            # 엔진을 묶은 그 파이썬 환경에서 실행
결과: app/src-tauri/resources/THIRD_PARTY_NOTICES.txt

- Python: 지금 환경에 설치된 패키지 (엔진에 같이 묶이는 것들)
- JavaScript: app의 배포 의존성 (npm ls --omit=dev)
- Rust: 데스크톱 셸 의존성 (cargo metadata)
- 함께 넣는 외부 도구: FFmpeg, Tesseract, 글꼴
"""
from __future__ import annotations

import json
import subprocess
import sys
from importlib import metadata
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
APP = ROOT / "app"
TAURI = APP / "src-tauri"
OUT = TAURI / "resources" / "THIRD_PARTY_NOTICES.txt"
LICENSE_NAMES = ("LICENSE", "LICENCE", "COPYING", "NOTICE", "AUTHORS")
SKIP_PY = {"pip", "setuptools", "wheel", "pyinstaller", "pyinstaller-hooks-contrib", "pytest", "localbiseo-engine", "altgraph", "pefile", "mach

BUNDLED_TOOLS = """\
FFmpeg (Windows 설치 파일에 포함)
  라이선스: GNU LGPL 2.1 이상 (LGPL 빌드, https://github.com/BtbN/FFmpeg-Builds)
  소스 코드: https://ffmpeg.org/download.html
  FFmpeg는 별도 실행 파일로 들어 있으며, 같은 버전의 다른 LGPL 빌드로 바꿔 쓸 수 있습니다.

Tesseract OCR과 한국어 학습 데이터 (Windows 설치 파일에 포함)
  라이선스: Apache License 2.0
  소스 코드: https://github.com/tesseract-ocr/tesseract , https://github.com/tesseract-ocr/tessdata_fast

나눔고딕 (워터마크 글꼴)
  라이선스: SIL Open Font License 1.1 (https://help.naver.com/service/30016/contents/18088)

처음 사용할 때 내려받는 AI 모델 (설치 파일에는 들어 있지 않음)
  Whisper (faster-whisper 변환본, Systran): MIT License
  U²-Net (rembg 배경 제거): Apache License 2.0
"""


def license_texts(paths: list[Path], limit: int = 40_000) -> str:
    chunks = []
    for p in paths:
        try:
            text = p.read_text(encoding="utf-8", errors="replace").strip()
        except OSError:
            continue
        if text and text not in chunks:
            chunks.append(text[:limit])
    return "\n\n".join(chunks)


def python_section() -> list[str]:
    out = []
    seen = set()
    for dist in sorted(metadata.distributions(), key=lambda d: (d.metadata["Name"] or "").lower()):
        name = dist.metadata["Name"] or ""
        if not name or name.lower() in seen or name.lower() in SKIP_PY:
```

```
            continue
        seen.add(name.lower())
        lic = dist.metadata.get("License-Expression") or dist.metadata.get("License") or ""
        if len(lic) > 120 or not lic:
            classifiers = [c.split("::")[-1].strip() for c in dist.metadata.get_all("Classifier") or [] if c.startswith("License ::")]
            lic = ", ".join(classifiers) or lic[:120]
        files = [Path(dist.locate_file(f)) for f in (dist.files or []) if Path(str(f)).name.upper().startswith(LICENSE_NAMES)]
        out.append(f"{name} {dist.version}\n  라이선스: {lic or '패키지 정보 참고'}\n\n{license_texts(files)}")
    return out


def npm_section() -> list[str]:
    try:
        raw = subprocess.run(["npm", "ls", "--omit=dev", "--all", "--json"], cwd=APP, capture_output=True, text=True, shell=sys.platform == "wi
        tree = json.loads(raw or "{}")
    except (OSError, json.JSONDecodeError):
        return []
    found: dict[str, str] = {}

    def walk(deps: dict) -> None:
        for name, info in (deps or {}).items():
            key = f"{name}@{info.get('version', '')}"
            if key not in found:
                found[key] = name
                walk(info.get("dependencies", {}))

    walk(tree.get("dependencies", {}))
    out = []
    for key, name in sorted(found.items()):
        pkg_dir = APP / "node_modules" / name
        try:
            pkg = json.loads((pkg_dir / "package.json").read_text(encoding="utf-8"))
        except (OSError, json.JSONDecodeError):
            continue
        files = [p for p in pkg_dir.iterdir() if p.is_file() and p.name.upper().startswith(LICENSE_NAMES)]
        out.append(f"{key}\n  라이선스: {pkg.get('license', '패키지 정보 참고')}\n\n{license_texts(files)}")
    return out


def cargo_section() -> list[str]:
    try:
        raw = subprocess.run(["cargo", "metadata", "--format-version", "1", "--locked"], cwd=TAURI, capture_output=True, text=True).stdout
        meta = json.loads(raw)
    except (OSError, json.JSONDecodeError):
        return []
    resolved = {n["id"] for n in (meta.get("resolve") or {}).get("nodes", [])}
    out = []
    for pkg in sorted(meta["packages"], key=lambda p: p["name"]):
        if pkg["id"] not in resolved or pkg.get("source") is None:
            continue
        d = Path(pkg["manifest_path"]).parent
        files = [p for p in d.iterdir() if p.is_file() and p.name.upper().startswith(LICENSE_NAMES)]
        # 같은 라이선스 본문이 수백 번 반복되지 않도록 Rust 쪽은 첫 파일만 넣는다.
        out.append(f"{pkg['name']} {pkg['version']}\n  라이선스: {pkg.get('license') or pkg.get('license_file') or '패키지 정보 참고'}\n\n{license_t
    return out


def main() -> None:
    sections = [
        ("함께 들어 있는 외부 도구와 글꼴", [BUNDLED_TOOLS]),
        ("엔진 (Python 패키지)", python_section()),
        ("화면 (JavaScript 패키지)", npm_section()),
```

```
        ("데스크톱 셸 (Rust 패키지)", cargo_section()),
    ]
    rule = "=" * 78
    parts = [
        "로컬비서 오픈소스 고지 (THIRD PARTY NOTICES)",
        "",
        "로컬비서는 아래 오픈소스 소프트웨어를 사용합니다. 각 소프트웨어의 저작권은 원저작자에게 있으며,",
        "각자의 라이선스 조건에 따라 사용합니다.",
        "",
    ]
    for title, items in sections:
        parts += [rule, f"■ {title} ({len(items)}개)", rule, ""]
        for item in items:
            parts += [item.strip(), "", "-" * 78, ""]
    OUT.parent.mkdir(parents=True, exist_ok=True)
    OUT.write_text("\n".join(parts), encoding="utf-8")
    counts = ", ".join(f"{t}: {len(i)}" for t, i in sections)
    print(f"{OUT} ({OUT.stat().st_size // 1024} KB) — {counts}")


if __name__ == "__main__":
    main()
```